

MA540 Project 5

Jeremy Welty

April 2026

Problem 1

Statement

We consider the function

$$f(q) = (6q^2 + 3)\sin(6q - 4)$$

for $q \in [0, 2]$. We will generate $M = 15$ uniform random samples $q^m \sim U(0, 2)$ and 15 Latin hypercube samples using `lhsdesign.m`. We will then plot $f(q)$ on $[0, 2]$ along with the two sets of samples $\{(q^m, f(q^m))\}_{m=1}^M$, and discuss the behavior of each. We will then use least-squares regression to construct an 8^{th} -order polynomial surrogate $f_S^K(q)$ using the Latin hypercube samples in `qdata.txt`, and compare to $f(q)$ for $q \in [0, 2]$ and $q \in [-0.5, 2.5]$. We will discuss the results of using a polynomial surrogate for extrapolatory or out-of-data predictions.

Solution

First, we use `unifrnd` to generate 15 uniform random samples in $(0, 2)$, and `lhsdesign.m` to generate 15 Latin hypercube samples. We then compute our function on each of these samples, as well as on an evenly distributed set of points on $[0, 2]$ for plotting.

Figure 1 shows the plots of each sample compared to the function plot. We see that since the uniform samples are random, so there is some clustering of points. On the other hand, the hypercube samples are spread more evenly, as the domain has been split into 15 subintervals, with one point in each. This means that the hypercube samples do a better job of covering the domain compared to the uniform samples.

Next, we construct a least-squares error function for an 8^{th} order polynomial, using the backslash operator in MATLAB to solve a linear system using the data in `qdata.txt`. We find $[\alpha_0, \alpha_1, \dots, \alpha_8] = 1.0 \times 10^3 * [0.0041, -0.0922, 0.7722, -3.3105, 7.0532, -7.7812, 4.5315, -1.3202, 0.1513]$. We then compute this polynomial surrogate $f_S^K(q)$ and $f(q)$ on an evenly distributed set of points on $[-0.5, 2.5]$.

Figure 2 shows the plots of the surrogate and $f(q)$. On $q \in [0, 2]$, the surrogate very closely matches $f(q)$, but the surrogate quickly deviates on $q \in [-0.5, 0] \cup [2, 2.5]$, the out-of-data predictions. Due to the nature of polynomials, they very closely wrap training data to interpolate the points, but shoot off at the ends, leading to highly inaccurate predictions outside the original training data domain.

Problem 2

Statement

We consider the spring model

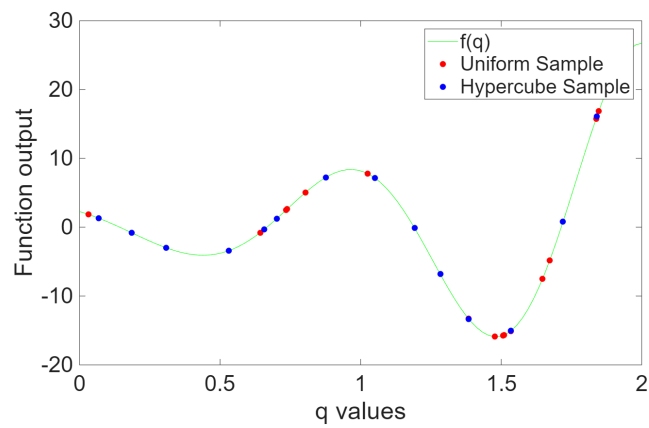


Figure 1: Uniform and Hypercube Samples

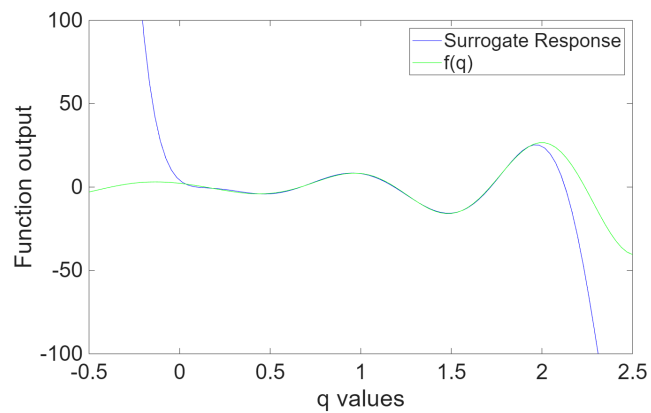


Figure 2: Polynomial Surrogate vs Function f

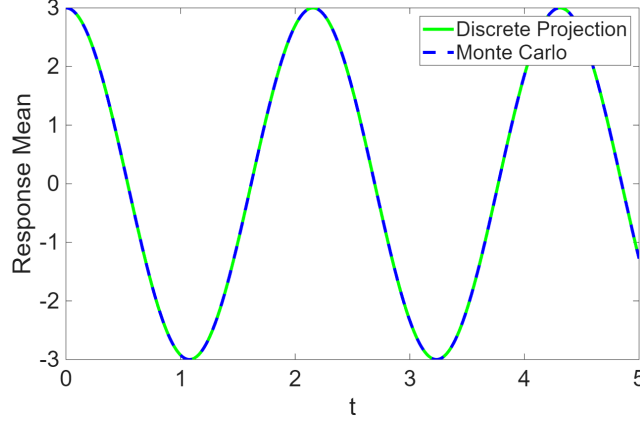


Figure 3: Discrete Projection Mean vs MC Mean

$$\frac{d^2 z}{dt^2} + kz = 0$$

$$z(0) = 3, \frac{dz}{dt}(0) = 0$$

with the solution $z(t, k) = 3 \cos(\sqrt{k} \cdot t)$. We consider $k \sim U(\bar{k} - \sigma_k, \bar{k} + \sigma_k)$ with $\bar{k} = 8.5$ and $\sigma_k = 0.001$, for model response $f(t, k) = z(t, k)$. We construct a Legendre surrogate $f_s^K(t, \xi)$ using $K + 1 = 4$ basis functions $\{\psi_k(\xi)\}_{k=0}^K$ with $\xi \in [-1, 1]$. We will compare the mean and standard deviation

$$\mathbb{E}[f_s^K(t, \xi)] = u_0(t)$$

$$\sqrt{\text{var}[f_s^K(t, \xi)]} = [\sum_{k=1}^K u_k^2(t) \gamma_k]^{1/2}$$

on $t \in [0, 5]$ with the Monte Carlo approximations

$$\mathbb{E}[f(t, k)] = \frac{1}{M} \sum_{m=1}^M f(t, k^m),$$

$$\sqrt{\text{var}[f(t, k)]} = [\frac{1}{M-1} \sum_{m=1}^M [f(t, k^m) - \mathbb{E}[f(t, k)]]^2]^{1/2}$$

computed using $M = 10^5$ samples. We will use the $R = 10$ Legendre quadrature points and weights in the MATLAB code for Example 16.15 when computing the coefficients $u_k(t)$.

Solution

Using `rand`, we construct $M = 10^5$ samples on $k \sim U(\bar{k} - \sigma_k, \bar{k} + \sigma_k)$, and construct an evenly distributed set of points $t \in [0, 5]$. Using the weights and quadrature points from Example 16.15, we compute the Monte Carlo and discrete projection values at each t-value for $M = 10^5$ iterations. We then compute the discrete projection mean and standard deviation. Figure 3 shows the discrete projection mean and Monte Carlo mean on $t \in [0, 5]$. As we can see, the discrete projection follows the MC results very closely.

Figure 4 shows the standard deviation for discrete projection and MC on $t \in [0, 5]$. Similarly, the discrete projection follows the MC results very closely, and we note the standard deviation follows the phase drift we'd expect for this function.

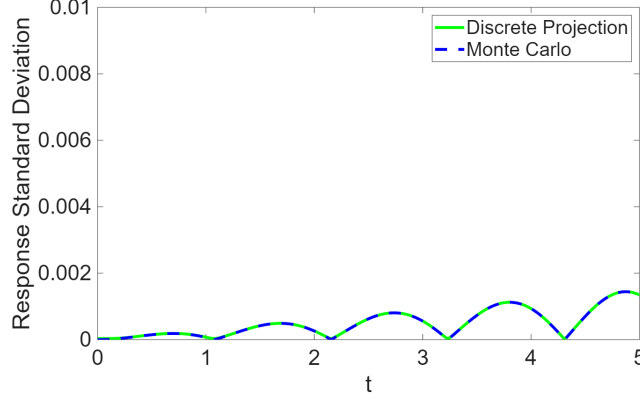


Figure 4: Discrete Projection SD vs MC SD

Problem 3

Statement

We consider the same function from Problem 1,

$$f(q) = (6q^2 + 3) \sin(6q - 4)$$

and the $M = 15$ Latin hypercube samples $q^m \in [0, 2]$ from `qdata.txt`. We will employ `fitrgp.m` to construct a Gaussian process (GP) surrogate using the option for a squared exponential kernel. We will plot the training points, GP mean, and GP prediction interval for $q \in [0, 2]$ and $q \in [-0.5, 2.5]$, and discuss.

Solution

We compute the function on the 15 samples from `qdata.txt`, and employ `fitrgp` to construct a GP surrogate based on the samples and function output from the samples, using a squared exponential kernel. We then use `predict` to compute model responses and 95% prediction intervals for our surrogate on an evenly distributed set of points $q \in [-0.5, 2.5]$. Figure 5 shows the training points, GP mean, and GP prediction intervals on $q \in [-0.5, 2.5]$, as well as $f(q)$ on the same interval. We also supply a zoomed version for a clearer visualization of the prediction intervals and model behavior.

The training points are all within the prediction intervals, as we expect. The GPR mean also closely matches $f(q)$ inside the training data domain, and thus would be a sufficient model for extrapolation within the same domain. We see that the prediction intervals are tight on $q \in [0, 2]$, but widen outside the original training domain as uncertainty grows. These intervals are around the GPR mean, so as that separates from the true value, the uncertainty compounds, and we see the true $f(q)$ slip outside the prediction intervals. This shows how points outside the training domain are a limitation of using a Gaussian process emulator for extrapolatory predictions. Due to the specification of the training, there is much greater uncertainty in these out-of-data predictions, compared to unobserved points within the original training domain.

Figure 5: Gaussian Process Mean and 95% Prediction Intervals with Training Points

